AGRICULTURE AI - FRONTEND DOCUMENTATION

TABLE OF CONTENTS

---

1. FRONTEND OVERVIEW

What is the Frontend?

The Agriculture AI frontend is a **React-based Single Page Application (SPA)** that provides an intuitive interface for farmers to manage their agricultural operations.

Key Characteristics:

- **Framework**: React 18.x with Hooks
- **Language**: JavaScript (ES6+)
- **Build Tool**: Create React App (Webpack)
- **Styling**: Tailwind CSS (Utility-first)
- **Routing**: React Router v6
- **State**: React Context API
- **i18n**: i18next (Multi-language)

Application Flow:

```
Browser Opens
    ↓
Check localStorage['languageSelected']
    ↓
No → Language Selector Screen
Yes → Check Authentication
    ↓
```

```
Not Authenticated → Login/Register
Authenticated → Dashboard
    ↓
User Interacts with Features
```

---

2. TECHNOLOGY STACK

Core Technologies

React 18.x

```
"react": "^18.2.0",
"react-dom": "^18.2.0"
```

Why React?

- Component-based architecture
- Virtual DOM for performance
- Large ecosystem
- Easy to learn and maintain

React Router v6

```
"react-router-dom": "^6.8.0"
```

Features Used:

- BrowserRouter for HTML5 history
- Routes and Route components
- Navigate for redirects
- useNavigate hook
- Protected routes pattern

Tailwind CSS

```
"tailwindcss": "^3.2.4"
```

Benefits:

- Utility-first approach
- Responsive by default
- Dark mode support

- Custom configuration
- No CSS file management

UI Libraries

i18next & react-i18next

```
"i18next": "^22.4.9",
"react-i18next": "^12.1.4"
```

Purpose: Multi-language support **Languages:** English, Hindi, Gujarati **Features:**

- Automatic language detection
- Translation key nesting
- Pluralization support
- LocalStorage persistence

Axios

```
"axios": "^1.3.2"
```

Purpose: HTTP client for API calls **Features:**

- Request/response interceptors
- Automatic JWT token injection
- Error handling
- Base URL configuration

Socket.IO Client

```
"socket.io-client": "^4.5.4"
```

Purpose: Real-time notifications **Features:**

- WebSocket connection
- Event-based communication
- Automatic reconnection
- Room support

Chart.js & react-chartjs-2

```
"chart.js": "^4.2.0",
"react-chartjs-2": "^5.2.0"
```

Purpose: Data visualization **Chart Types Used:**

- Bar charts (crop health)
- Pie charts (soil distribution)
- Line charts (monthly trends)
- Doughnut charts (crop types)

react-share

```
"react-share": "^4.4.1"
```

Purpose: Social media sharing **Platforms:**

- WhatsApp
- Twitter
- Facebook
- Telegram

---

3. PROJECT STRUCTURE

Complete Directory Tree

```
client/
├── public/
│   ├── index.html                # HTML template
│   ├── manifest.json             # PWA manifest
│   └── favicon.ico               # App icon
│
├── src/
│   ├── components/               # All React components
│   │   ├── Analytics/
│   │   │   └── Analytics.js      # Charts & insights
│   │   ├── Auth/
│   │   │   ├── Login.js          # Login form ☑
│   │   │   └── Register.js       # Registration form ☑
│   │   ├── Chatbot/
│   │   │   └── Chatbot.js        # AI assistant interface
│   │   ├── Crops/
│   │   │   └── Crops.js          # Crop management
│   │   ├── Dashboard/
│   │   │   └── Dashboard.js      # Main dashboard ☑
│   │   ├── DiseaseDetection/
│   │   │   └── DiseaseDetection.js  # Disease AI
│   │   ├── Farms/
│   │   │   └── Farms.js          # Farm management ☑☑
│   │   ├── LanguageSelector/
│   │   │   └── LanguageSelector.js  # First visit ☑
```

```
│   │   ├── LanguageSwitcher/
│   │   │   └── LanguageSwitcher.js  # Header dropdown ☑
│   │   ├── Layout/
│   │   │   └── Layout.js         # Sidebar & header ☑
│   │   ├── Market/
│   │   │   └── Market.js         # Market prices
│   │   ├── Notifications/
│   │   │   └── NotificationBell.js  # Bell icon
│   │   ├── Profile/
│   │   │   └── Profile.js        # User profile
│   │   ├── Recommendations/
│   │   │   └── Recommendations.js   # AI suggestions
│   │   ├── Schemes/
│   │   │   └── Schemes.js        # Govt schemes
│   │   ├── ShareButton/
│   │   │   └── ShareButton.js    # Social share
│   │   └── Weather/
│   │       └── Weather.js        # Weather widget
│   │
│   ├── context/
│   │   └── AuthContext.js        # Global auth state ☑
│   │
│   ├── i18n/
│   │   ├── i18n.js               # i18next config ☑
│   │   └── locales/
│   │       ├── en.json           # English ☑
│   │       ├── hi.json           # Hindi ☑
│   │       └── gu.json           # Gujarati ☑
│   │
│   ├── services/
│   │   ├── api.js                # Axios instance ☑
│   │   └── socket.js             # Socket.IO setup
│   │
│   ├── utils/
│   │   ├── exportCSV.js          # CSV export ☑☑
│   │   ├── pdfExport.js          # (deprecated)
│   │   └── simplePDFExport.js    # (deprecated)
│   │
│   ├── App.js                    # Main app component ☑
│   ├── index.js                  # Entry point ☑
│   ├── index.css                 # Global styles
│   ├── service-worker.js         # PWA support
│   └── serviceWorkerRegistration.js
│
├── .env                          # Environment variables
├── .env.example                  # Env template
├── .eslintrc.json               # ESLint config
├── package.json                  # Dependencies
├── postcss.config.js            # PostCSS config
├── tailwind.config.js           # Tailwind config
└── README.md

☑ = Fully translated
☑☑ = Fully translated + Export working
```

---

4. COMPONENT ARCHITECTURE

Component Hierarchy

```
App (Root)
├── LanguageSelector (First visit only)
└── Router
    ├── Public Routes
    │   ├── Login
    │   └── Register
    └── Protected Routes
        └── Layout (Sidebar + Header)
            ├── Dashboard
            ├── Farms ☑☑
            ├── Crops
            ├── DiseaseDetection
            ├── Chatbot
            ├── Recommendations
            ├── Weather
            ├── Market
            ├── Schemes
            ├── Profile
            └── Analytics
```

Component Types

1. Container Components (Smart)

Purpose: Handle logic, state, and API calls

Example: Farms.js

```
const Farms = () => {
  // State
  const [farms, setFarms] = useState([]);
  const [loading, setLoading] = useState(true);

  // API calls
  useEffect(() => {
    fetchFarms();
  }, []);

  // Business logic
  const handleSubmit = async (e) => {
    // Form submission logic
  };
```

```
  // Render UI
  return <div>...</div>;
};
```

2. Presentational Components (Dumb)

Purpose: Only display UI, receive props

Example: FarmCard (if separated)

```
const FarmCard = ({ farm, onEdit, onDelete }) => {
  return (
    <div className="card">
      <h3>{farm.farmName}</h3>
      <button onClick={() => onEdit(farm)}>Edit</button>
      <button onClick={() => onDelete(farm._id)}>Delete</button>
    </div>
  );
};
```

3. Layout Components

Purpose: Structure and navigation

Layout.js Structure:

```
const Layout = ({ children }) => {
  return (
    <div className="flex">
      <Sidebar />
      <div className="main-content">
        <Header />
        <main>{children}</main>
      </div>
    </div>
  );
};
```

4. Feature Components

Purpose: Specific feature implementation

Examples:

- `LanguageSelector.js` - First visit language choice
- `NotificationBell.js` - Real-time notifications
- `ShareButton.js` - Social media sharing

- `LanguageSwitcher.js` - Header language dropdown

---

5. STATE MANAGEMENT

React Context API

AuthContext - Global Authentication State

File: `src/context/AuthContext.js`

Structure:

```
const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [loading, setLoading] = useState(true);

  // Check authentication on mount
  useEffect(() => {
    checkAuth();
  }, []);

  const checkAuth = async () => {
    const token = localStorage.getItem('token');
    if (token) {
      try {
        const response = await api.get('/auth/me');
        setUser(response.data.data);
        setIsAuthenticated(true);
      } catch (error) {
        localStorage.removeItem('token');
      }
    }
    setLoading(false);
  };

  const login = async (phone, password) => {
    try {
      const response = await api.post('/auth/login', { phone, password });
      const { token, user } = response.data.data;

      localStorage.setItem('token', token);
      setUser(user);
      setIsAuthenticated(true);

      return { success: true };
    } catch (error) {
      return {
        success: false,
```

```
        message: error.response?.data?.message || 'Login failed'
      };
    }
  };

  const register = async (userData) => {
    // Similar to login
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
    setIsAuthenticated(false);
  };

  return (
    <AuthContext.Provider value={{
      user,
      isAuthenticated,
      loading,
      login,
      register,
      logout
    }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);
```

Usage in Components:

```
import { useAuth } from '../../context/AuthContext';

function MyComponent() {
  const { user, isAuthenticated, logout } = useAuth();

  return (
    <div>
      <p>Welcome, {user?.name}</p>
      <button onClick={logout}>Logout</button>
    </div>
  );
}
```

Local State (useState)

Common Patterns:

1. Form State

```
const [formData, setFormData] = useState({
  farmName: '',
  area: '',
  location: { state: '', district: '' }
});

// Update single field
const handleChange = (e) => {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};

// Update nested field
const handleLocationChange = (field, value) => {
  setFormData({
    ...formData,
    location: { ...formData.location, [field]: value }
  });
};
```

2. Loading State

```
const [loading, setLoading] = useState(true);

useEffect(() => {
  fetchData();
}, []);

const fetchData = async () => {
  setLoading(true);
  try {
    const response = await api.get('/farms');
    setFarms(response.data.data);
  } finally {
    setLoading(false);
  }
};

if (loading) return <Spinner />;
```

3. Modal/Dialog State

```
const [showForm, setShowForm] = useState(false);
const [editingFarm, setEditingFarm] = useState(null);

const openEditForm = (farm) => {
```

```
    setEditingFarm(farm);
    setShowForm(true);
  };

  const closeForm = () => {
    setEditingFarm(null);
    setShowForm(false);
  };
```

---

6. ROUTING SYSTEM

React Router v6 Implementation

File: `src/App.js`

Route Structure

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-
dom';

function App() {
  return (
    <Router>
      <AuthProvider>
        <Routes>
          {/* Public Routes */}
          <Route path="/login" element={<PublicRoute><Login /></PublicRoute>} />
          <Route path="/register" element={<PublicRoute><Register />
</PublicRoute>} />

          {/* Protected Routes */}
          <Route path="/dashboard" element={
            <ProtectedRoute>
              <Layout><Dashboard /></Layout>
            </ProtectedRoute>
          } />

          <Route path="/farms" element={
            <ProtectedRoute>
              <Layout><Farms /></Layout>
            </ProtectedRoute>
          } />

          {/* Redirect root to dashboard */}
          <Route path="/" element={<Navigate to="/dashboard" />} />

          {/* Catch all - redirect to dashboard */}
          <Route path="*" element={<Navigate to="/dashboard" />} />
        </Routes>
      </AuthProvider>
```

```
    </Router>
  );
}
```

Protected Route Component

```
const ProtectedRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return (
      <div className="flex items-center justify-center min-h-screen">
        <div className="animate-spin rounded-full h-16 w-16 border-b-2 border-
green-600"></div>
      </div>
    );
  }

  return isAuthenticated ? children : <Navigate to="/login" />;
};
```

Public Route Component

```
const PublicRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return <Spinner />;
  }

  // Redirect to dashboard if already authenticated
  return isAuthenticated ? <Navigate to="/dashboard" /> : children;
};
```

Navigation Hooks

useNavigate

```
import { useNavigate } from 'react-router-dom';

function Login() {
  const navigate = useNavigate();

  const handleLogin = async () => {
    const result = await login(phone, password);
    if (result.success) {
```

```
        navigate('/dashboard');
      }
    };
  }
```

Link Component

```
import { Link } from 'react-router-dom';

<Link to="/farms" className="nav-link">
  My Farms
</Link>
```

Route Parameters

```
// Not currently used, but here's how:
<Route path="/farms/:id" element={<FarmDetail />} />

// In component:
import { useParams } from 'react-router-dom';
const { id } = useParams();
```

---

7. MULTI-LANGUAGE IMPLEMENTATION

i18next Configuration

File: `src/i18n/i18n.js`

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import en from './locales/en.json';
import hi from './locales/hi.json';
import gu from './locales/gu.json';

i18n
  .use(initReactI18next)
  .init({
    resources: {
      en: { translation: en },
      hi: { translation: hi },
      gu: { translation: gu }
    },
    lng: localStorage.getItem('language') || 'en',
    fallbackLng: 'en',
    interpolation: {
```

```
      escapeValue: false  // React already escapes
    }
  });

export default i18n;
```

Translation Files Structure

File: `src/i18n/locales/en.json`

```
{
  "common": {
    "welcome": "Welcome",
    "logout": "Logout",
    "save": "Save",
    "cancel": "Cancel",
    "delete": "Delete",
    "edit": "Edit"
  },
  "nav": {
    "dashboard": "Dashboard",
    "farms": "My Farms",
    "crops": "My Crops"
  },
  "farms": {
    "title": "My Farms",
    "subtitle": "Manage your farms and track their details",
    "addFarm": "Add New Farm",
    "farmName": "Farm Name",
    "area": "Area (acres)",
    "state": "State",
    "district": "District",
    "village": "Village"
  }
}
```

Hindi Translation: `src/i18n/locales/hi.json`

```
{
  "common": {
    "welcome": "स्वागत",
    "logout": "लॉग आउट",
    "save": "सहेजें",
    "cancel": "रद्द करें"
  },
  "farms": {
    "title": "मेरे खेत",
    "subtitle": "अपने खेतों का प्रबंधन करें",
    "addFarm": "नया खेत जोड़ें"
```

```
  }
}
```

Using Translations in Components

Basic Usage

```
import { useTranslation } from 'react-i18next';

function MyComponent() {
  const { t } = useTranslation();

  return (
    <div>
      <h1>{t('farms.title')}</h1>
      <p>{t('farms.subtitle')}</p>
      <button>{t('common.save')}</button>
    </div>
  );
}
```

With Fallback

```
<h2>{t('farms.title') || 'My Farms'}</h2>
```

With Variables

```
// Translation file
{
  "greeting": "Hello, {{name}}!"
}

// Component
{t('greeting', { name: user.name })}
```

Changing Language

```
import { useTranslation } from 'react-i18next';

function LanguageSwitcher() {
  const { i18n } = useTranslation();

  const changeLanguage = (lang) => {
```

```
    i18n.changeLanguage(lang);
    localStorage.setItem('language', lang);
  };

  return (
    <div>
      <button onClick={() => changeLanguage('en')}>English</button>
      <button onClick={() => changeLanguage('hi')}>हिंदी</button>
      <button onClick={() => changeLanguage('gu')}>ગુજરાતી</button>
    </div>
  );
}
```

Language Selector Component

File: `src/components/LanguageSelector/LanguageSelector.js`

```
import React from 'react';
import { useTranslation } from 'react-i18next';

const LanguageSelector = ({ onLanguageSelect }) => {
  const { i18n } = useTranslation();

  const languages = [
    { code: 'en', name: 'English', nativeName: 'English', flag: 'GB' },
    { code: 'hi', name: 'Hindi', nativeName: 'हिंदी', flag: 'IN' },
    { code: 'gu', name: 'Gujarati', nativeName: 'ગુજરાતી', flag: 'IN' }
  ];

  const selectLanguage = (code) => {
    i18n.changeLanguage(code);
    localStorage.setItem('language', code);
    localStorage.setItem('languageSelected', 'true');
    if (onLanguageSelect) onLanguageSelect(code);
  };

  return (
    <div className="min-h-screen bg-gradient-to-br from-green-50 to-blue-50 flex
items-center justify-center">
      <div className="bg-white rounded-2xl shadow-2xl p-8 max-w-md w-full">
        <div className="text-center mb-8">
          <div className="text-6xl mb-4">🌾</div>
          <h1 className="text-3xl font-bold text-gray-800 mb-2">
            Agriculture AI
          </h1>
          <p className="text-gray-600 text-sm">Smart Farming System</p>
        </div>

        <div className="mb-6">
          <h2 className="text-xl font-bold text-center text-gray-800 mb-2">
            Choose Language
```

```
          </h2>
          <p className="text-center text-sm text-gray-600 mb-1">भाषा चुनें</p>
          <p className="text-center text-sm text-gray-600">ભાષા પસંદ કરો</p>
        </div>

        <div className="space-y-3">
          {languages.map((lang) => (
            <button
              key={lang.code}
              onClick={() => selectLanguage(lang.code)}
              className="w-full flex items-center justify-between p-4 rounded-xl
border-2 border-gray-200 hover:border-green-500 hover:bg-green-50 transition-all"
            >
              <div className="flex items-center space-x-4">
                <span className="text-4xl">{lang.flag}</span>
                <div className="text-left">
                  <div className="font-semibold text-gray-800">{lang.name}</div>
                  <div className="text-sm text-gray-600">{lang.nativeName}</div>
                </div>
              </div>
              <svg className="w-6 h-6 text-gray-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M9 5l7 7-7 7" />
              </svg>
            </button>
          ))}
        </div>
      </div>
    </div>
  );
};

export default LanguageSelector;
```

First-Visit Language Selection

In App.js:

```
function App() {
  const [languageSelected, setLanguageSelected] = useState(false);

  useEffect(() => {
    const selected = localStorage.getItem('languageSelected');
    if (selected === 'true') {
      setLanguageSelected(true);
    }
  }, []);

  const handleLanguageSelect = () => {
    setLanguageSelected(true);
```

```
  };

  // Show language selector on first visit
  if (!languageSelected) {
    return <LanguageSelector onLanguageSelect={handleLanguageSelect} />;
  }

  return <Router>...</Router>;
}
```

Translation Best Practices

1. **Always use t() function** - Never hardcode text
2. **Organize keys logically** - Group by feature
3. **Use nested keys** - `farms.title` not `farmsTitle`
4. **Provide fallbacks** - `t('key') || 'Default'`
5. **Test all languages** - Switch and verify
6. **Keep keys consistent** - Same structure across languages
7. **Document new keys** - Add to all language files

---

8. STYLING & UI

Tailwind CSS Configuration

File: `tailwind.config.js`

```
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx}",
  ],
  theme: {
    extend: {
      colors: {
        primary: {
          50: '#f0fdf4',
          100: '#dcfce7',
          500: '#22c55e',
          600: '#16a34a',
          700: '#15803d',
        }
      }
    },
  },
  plugins: [],
}
```

Common Utility Classes

Layout

```
<!-- Container -->
<div className="container mx-auto px-4">

<!-- Flexbox -->
<div className="flex items-center justify-between">

<!-- Grid -->
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">

<!-- Spacing -->
<div className="mt-4 mb-6 p-8">
```

Colors

```
<!-- Background -->
<div className="bg-white">
<div className="bg-green-50">
<div className="bg-gradient-to-r from-green-500 to-blue-600">

<!-- Text -->
<p className="text-gray-600">
<h1 className="text-green-700">
```

Typography

```
<h1 className="text-3xl font-bold">
<p className="text-sm text-gray-600">
<span className="font-medium">
```

Buttons

```
<!-- Primary Button -->
<button className="bg-green-600 text-white px-6 py-2 rounded-lg hover:bg-green-700
transition">
  Save
</button>

<!-- Secondary Button -->
<button className="bg-gray-300 text-gray-700 px-6 py-2 rounded-lg hover:bg-gray-
400">
  Cancel
</button>
```

```
<!-- Icon Button -->
<button className="p-2 rounded-full hover:bg-gray-100">
  <svg>...</svg>
</button>
```

Cards

```
<div className="bg-white rounded-lg shadow p-6 hover:shadow-lg transition">
  <h3 className="text-xl font-bold mb-4">Title</h3>
  <p className="text-gray-600">Content</p>
</div>
```

Forms

```
<!-- Input -->
<input
  type="text"
  className="w-full px-3 py-2 border rounded-lg focus:ring-2 focus:ring-green-500"
/>

<!-- Label -->
<label className="block text-sm font-medium mb-1">
  Farm Name
</label>

<!-- Select -->
<select className="w-full px-3 py-2 border rounded-lg">
  <option>Option 1</option>
</select>
```

Responsive Design

Breakpoints

```
sm: 640px   // Mobile landscape
md: 768px   // Tablet
lg: 1024px  // Desktop
xl: 1280px  // Large desktop
```

Usage

```
<!-- Mobile: 1 column, Tablet: 2 columns, Desktop: 3 columns -->
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3">

<!-- Hide on mobile, show on desktop -->
<div className="hidden lg:block">

<!-- Full width on mobile, half on desktop -->
<div className="w-full lg:w-1/2">
```

Custom CSS

File: src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Custom scrollbar */
::-webkit-scrollbar {
  width: 8px;
}

::-webkit-scrollbar-track {
  background: #f1f1f1;
}

::-webkit-scrollbar-thumb {
  background: #888;
  border-radius: 4px;
}

/* Custom animations */
@keyframes spin {
  to { transform: rotate(360deg); }
}

.spinner {
  animation: spin 1s linear infinite;
}

/* Custom card styles */
.card {
  @apply bg-white rounded-lg shadow p-6;
}

/* Custom button styles */
.btn-primary {
  @apply bg-green-600 text-white px-6 py-2 rounded-lg hover:bg-green-700
transition;
}
```

```
.btn-secondary {
  @apply bg-gray-300 text-gray-700 px-6 py-2 rounded-lg hover:bg-gray-400
transition;
}
```

Icons

Using inline SVG for icons:

```
// Menu Icon
<svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24">
  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M4 6h16M4
12h16M4 18h16" />
</svg>

// Close Icon
<svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M6 18L18
6M6 6l12 12" />
</svg>

// Download Icon
<svg className="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M12 10v6m0
0l-3-3m3 3l3-3m2 8H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1 1 0 01.707.293l5.414
5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z" />
</svg>
```

---

9. API INTEGRATION

Axios Configuration

File: src/services/api.js

```
import axios from 'axios';

// Create axios instance with base configuration
const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:5000/api',
  headers: {
    'Content-Type': 'application/json'
  },
  timeout: 10000  // 10 seconds
});

// Request Interceptor - Add JWT token to all requests
api.interceptors.request.use(
  (config) => {
```

```
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Response Interceptor - Handle errors globally
api.interceptors.response.use(
  (response) => {
    return response;
  },
  (error) => {
    // Handle 401 Unauthorized - redirect to login
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }

    // Handle network errors
    if (!error.response) {
      console.error('Network Error:', error);
    }

    return Promise.reject(error);
  }
);

export default api;

// Specific API modules
export const farmAPI = {
  getAll: () => api.get('/farms'),
  getById: (id) => api.get(`/farms/${id}`),
  create: (data) => api.post('/farms', data),
  update: (id, data) => api.put(`/farms/${id}`, data),
  delete: (id) => api.delete(`/farms/${id}`),
  getStats: () => api.get('/farms/stats')
};

export const cropAPI = {
  getAll: () => api.get('/crops'),
  create: (data) => api.post('/crops', data),
  update: (id, data) => api.put(`/crops/${id}`, data),
  delete: (id) => api.delete(`/crops/${id}`),
  getStats: () => api.get('/crops/stats')
};

export const weatherAPI = {
  getCurrent: (params) => api.get('/weather/current', { params }),
```

```
  getForecast: (params) => api.get('/weather/forecast', { params })
};

export const chatAPI = {
  sendMessage: (data) => api.post('/chat/message', data),
  getDailyTip: () => api.get('/chat/daily-tip')
};
```

API Usage in Components

Example: Fetching Data

```
import { useState, useEffect } from 'react';
import { farmAPI } from '../../services/api';

function Farms() {
  const [farms, setFarms] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetchFarms();
  }, []);

  const fetchFarms = async () => {
    setLoading(true);
    setError(null);
    try {
      const response = await farmAPI.getAll();
      setFarms(response.data.data || []);
    } catch (err) {
      console.error('Failed to fetch farms:', err);
      setError(err.response?.data?.message || 'Failed to load farms');
    } finally {
      setLoading(false);
    }
  };

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return <div>{/* Render farms */}</div>;
}
```

Example: Creating Data

```
const handleSubmit = async (e) => {
  e.preventDefault();
```

```
  try {
    const response = await farmAPI.create(formData);

    // Success
    setFarms([...farms, response.data.data]);
    setShowForm(false);
    alert('Farm created successfully!');

  } catch (error) {
    // Error handling
    const message = error.response?.data?.message || 'Failed to create farm';
    alert(message);
  }
};
```

Example: Updating Data

```
const handleUpdate = async (id, updatedData) => {
  try {
    const response = await farmAPI.update(id, updatedData);

    // Update local state
    setFarms(farms.map(farm =>
      farm._id === id ? response.data.data : farm
    ));

    alert('Farm updated successfully!');
  } catch (error) {
    alert('Failed to update farm');
  }
};
```

Example: Deleting Data

```
const handleDelete = async (id) => {
  if (!window.confirm('Are you sure you want to delete this farm?')) {
    return;
  }

  try {
    await farmAPI.delete(id);

    // Remove from local state
    setFarms(farms.filter(farm => farm._id !== id));

    alert('Farm deleted successfully!');
  } catch (error) {
```

```
      alert('Failed to delete farm');
    }
  };
```

Error Handling Patterns

Try-Catch Pattern

```
try {
  const response = await api.get('/endpoint');
  // Success handling
} catch (error) {
  if (error.response) {
    // Server responded with error
    console.error('Server Error:', error.response.data);
    alert(error.response.data.message);
  } else if (error.request) {
    // Request made but no response
    console.error('Network Error:', error.request);
    alert('Network error. Please check your connection.');
  } else {
    // Other errors
    console.error('Error:', error.message);
    alert('An error occurred');
  }
}
```

Async/Await with Finally

```
const [loading, setLoading] = useState(false);

const fetchData = async () => {
  setLoading(true);
  try {
    const response = await api.get('/data');
    setData(response.data.data);
  } catch (error) {
    console.error(error);
  } finally {
    setLoading(false);  // Always runs
  }
};
```

Loading States

```
if (loading) {
  return (
    <div className="flex items-center justify-center min-h-screen">
      <div className="animate-spin rounded-full h-16 w-16 border-b-2 border-green-
600"></div>
    </div>
  );
}
```

---

10. FEATURES IMPLEMENTATION

Feature 1: Farms Management ☑☑ COMPLETE

File: `src/components/Farms/Farms.js`

Features:

- ☑ List all farms in grid layout
- ☑ Add new farm with form
- ☑ Edit existing farm
- ☑ Delete farm with confirmation
- ☑ Export to CSV
- ☑ Full translation (EN/HI/GU)
- ☑ Responsive design

Key Code Sections:

State Management

```
const [farms, setFarms] = useState([]);
const [loading, setLoading] = useState(true);
const [showForm, setShowForm] = useState(false);
const [editingFarm, setEditingFarm] = useState(null);
const [formData, setFormData] = useState({
  farmName: '',
  location: { state: '', district: '', village: '', pincode: '' },
  area: '',
  soil_type: 'Loamy',
  irrigation_type: 'Drip',
  water_source: 'Well'
});
```

Form Handling

```
const handleSubmit = async (e) => {
  e.preventDefault();
```

```
  try {
    if (editingFarm) {
      await api.put(`/farms/${editingFarm._id}`, formData);
    } else {
      await api.post('/farms', formData);
    }
    fetchFarms();
    resetForm();
  } catch (error) {
    alert(error.response?.data?.message || 'Failed to save farm');
  }
};
```

CSV Export

```
import { exportFarmsToCSV } from '../../utils/exportCSV';

const handleExportPDF = () => {
  if (farms.length === 0) {
    alert(t('farms.noFarmsToExport'));
    return;
  }

  const success = exportFarmsToCSV(farms, user?.name || 'User');

  if (success) {
    alert(t('farms.exportSuccess'));
  } else {
    alert(t('farms.exportFailed'));
  }
};
```

Translation Usage

```
const { t } = useTranslation();

return (
  <div>
    <h2>{t('farms.title')}</h2>
    <p>{t('farms.subtitle')}</p>
    <button>{t('farms.addFarm')}</button>
    <label>{t('farms.farmName')}</label>
  </div>
);
```

Feature 2: Dashboard

File: `src/components/Dashboard/Dashboard.js`

Features:

- Statistics cards (farms, crops, area)
- Weather widget
- Daily farming tip
- Quick action cards
- Partial translation

Statistics Cards:

```
const [stats, setStats] = useState(null);

useEffect(() => {
  fetchDashboardData();
}, []);

const fetchDashboardData = async () => {
  try {
    const farmStatsRes = await farmAPI.getStats();
    const cropStatsRes = await cropAPI.getStats();

    setStats({
      farms: farmStatsRes.data.data,
      crops: cropStatsRes.data.data
    });
  } catch (error) {
    console.error('Error fetching dashboard data:', error);
  }
};
```

Feature 3: Authentication

Login Component: `src/components/Auth/Login.js`

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import { useTranslation } from 'react-i18next';

const Login = () => {
  const navigate = useNavigate();
  const { login } = useAuth();
  const { t, i18n } = useTranslation();

  const [formData, setFormData] = useState({
    phone: '',
    password: ''
  });
```

```
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError('');

    const result = await login(formData.phone, formData.password);

    if (result.success) {
      navigate('/dashboard');
    } else {
      setError(result.message);
    }

    setLoading(false);
  };

  const changeLanguage = (lng) => {
    i18n.changeLanguage(lng);
    localStorage.setItem('language', lng);
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-
br from-primary-50 to-primary-100">
      {/* Language Switcher */}
      <div className="absolute top-4 right-4 flex space-x-2">
        <button onClick={() => changeLanguage('en')}>English</button>
        <button onClick={() => changeLanguage('hi')}>हिंदी</button>
        <button onClick={() => changeLanguage('gu')}>ગુજરાતી</button>
      </div>

      {/* Login Form */}
      <div className="max-w-md w-full bg-white rounded-lg shadow-lg p-8">
        <h1 className="text-3xl font-bold mb-6">Agriculture AI</h1>

        {error && (
          <div className="bg-red-100 text-red-700 p-3 rounded mb-4">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit}>
          <div className="mb-4">
            <label className="block mb-2">Phone Number</label>
            <input
              type="tel"
              value={formData.phone}
              onChange={(e) => setFormData({...formData, phone: e.target.value})}
              className="w-full px-3 py-2 border rounded-lg"
              required
            />
```

```
          </div>

          <div className="mb-6">
            <label className="block mb-2">Password</label>
            <input
              type="password"
              value={formData.password}
              onChange={(e) => setFormData({...formData, password:
e.target.value})}
              className="w-full px-3 py-2 border rounded-lg"
              required
            />
          </div>

          <button
            type="submit"
            disabled={loading}
            className="w-full bg-green-600 text-white py-2 rounded-lg hover:bg-
green-700"
          >
            {loading ? 'Logging in...' : 'Login'}
          </button>
        </form>

        <p className="mt-4 text-center">
          Don't have an account?{' '}
          <Link to="/register" className="text-green-600">Sign up</Link>
        </p>
      </div>
    </div>
  );
};
```

Feature 4: Layout (Sidebar + Header)

File: `src/components/Layout/Layout.js`

```
import { useState } from 'react';
import { Link, useLocation } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import { useTranslation } from 'react-i18next';
import LanguageSwitcher from '../LanguageSwitcher/LanguageSwitcher';
import NotificationBell from '../Notifications/NotificationBell';

const Layout = ({ children }) => {
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const { user, logout } = useAuth();
  const { t } = useTranslation();
  const location = useLocation();

  const menuItems = [
```

```
    { path: '/dashboard', icon: '📊', labelKey: 'nav.dashboard' },
    { path: '/analytics', icon: '📈', labelKey: 'nav.analytics' },
    { path: '/farms', icon: '🌾', labelKey: 'nav.farms' },
    { path: '/crops', icon: '🌱', labelKey: 'nav.crops' },
    { path: '/disease-detection', icon: '🔬', labelKey: 'nav.disease' },
    { path: '/chatbot', icon: '💬', labelKey: 'nav.chatbot' },
    { path: '/recommendations', icon: '💡', labelKey: 'nav.recommendations' },
    { path: '/weather', icon: '🌤️', labelKey: 'nav.weather' },
    { path: '/market', icon: '💰', labelKey: 'nav.market' },
    { path: '/schemes', icon: '📋', labelKey: 'nav.schemes' },
    { path: '/profile', icon: '👤', labelKey: 'nav.profile' }
  ];

  return (
    <div className="flex h-screen bg-gray-100">
      {/* Sidebar */}
      <aside className={`bg-white shadow-lg transition-all duration-300
${sidebarOpen ? 'w-64' : 'w-20'}`}>
        <div className="p-4">
          <h1 className="text-2xl font-bold text-green-600">
            {sidebarOpen ? 'Agriculture AI' : '🌾'}
          </h1>
        </div>

        <nav className="mt-6">
          {menuItems.map((item) => (
            <Link
              key={item.path}
              to={item.path}
              className={`flex items-center px-4 py-3 ${
                location.pathname === item.path
                  ? 'bg-green-50 text-green-600 border-r-4 border-green-600'
                  : 'text-gray-600 hover:bg-gray-50'
              }`}
            >
              <span className="text-2xl">{item.icon}</span>
              {sidebarOpen && (
                <span className="ml-3">{t(item.labelKey)}</span>
              )}
            </Link>
          ))}
        </nav>
      </aside>

      {/* Main Content */}
      <div className="flex-1 flex flex-col overflow-hidden">
        {/* Header */}
        <header className="bg-white shadow-sm">
          <div className="flex items-center justify-between px-6 py-4">
            <button
              onClick={() => setSidebarOpen(!sidebarOpen)}
              className="p-2 rounded-lg hover:bg-gray-100"
            >
              <svg className="w-6 h-6" fill="none" stroke="currentColor"
```

```
viewBox="0 0 24 24">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M4 6h16M4 12h16M4 18h16" />
              </svg>
            </button>

            <div className="flex items-center space-x-4">
              <NotificationBell />
              <LanguageSwitcher />
              <span className="text-gray-700">{user?.name}</span>
              <button
                onClick={logout}
                className="px-4 py-2 bg-red-500 text-white rounded-lg hover:bg-
red-600"
              >
                {t('common.logout')}
              </button>
            </div>
          </div>
        </header>

        {/* Page Content */}
        <main className="flex-1 overflow-y-auto p-6">
          {children}
        </main>
      </div>
    </div>
  );
};

export default Layout;
```

---

11. FORMS & VALIDATION

Form Patterns

Controlled Components

```
const [formData, setFormData] = useState({
  farmName: '',
  area: '',
  state: ''
});

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
```

```
};

<input
  name="farmName"
  value={formData.farmName}
  onChange={handleChange}
/>
```

Nested Object State

```
const handleLocationChange = (field, value) => {
  setFormData(prev => ({
    ...prev,
    location: {
      ...prev.location,
      [field]: value
    }
  }));
};

<input
  value={formData.location.state}
  onChange={(e) => handleLocationChange('state', e.target.value)}
/>
```

Client-Side Validation

Basic Validation

```
const validateForm = () => {
  const errors = {};

  if (!formData.farmName.trim()) {
    errors.farmName = 'Farm name is required';
  }

  if (!formData.area || formData.area <= 0) {
    errors.area = 'Area must be greater than 0';
  }

  if (!formData.location.state) {
    errors.state = 'State is required';
  }

  return errors;
};

const handleSubmit = async (e) => {
```

```
  e.preventDefault();

  const errors = validateForm();
  if (Object.keys(errors).length > 0) {
    setFormErrors(errors);
    return;
  }

  // Proceed with API call
};
```

Display Errors

```
{formErrors.farmName && (
  <p className="text-red-500 text-sm mt-1">
    {formErrors.farmName}
  </p>
)}
```

Form Reset

```
const resetForm = () => {
  setFormData({
    farmName: '',
    area: '',
    location: { state: '', district: '', village: '' }
  });
  setFormErrors({});
  setEditingFarm(null);
  setShowForm(false);
};
```

---

12. REAL-TIME FEATURES

Socket.IO Configuration

File: `src/services/socket.js`

```
import { io } from 'socket.io-client';

const SOCKET_URL = process.env.REACT_APP_SOCKET_URL || 'http://localhost:5000';

let socket = null;

export const initializeSocket = (userId) => {
```

```
  if (!socket) {
    socket = io(SOCKET_URL, {
      auth: {
        token: localStorage.getItem('token')
      }
    });

    socket.on('connect', () => {
      console.log('Socket connected:', socket.id);
      socket.emit('join', userId);
    });

    socket.on('disconnect', () => {
      console.log('Socket disconnected');
    });
  }

  return socket;
};

export const getSocket = () => socket;

export const disconnectSocket = () => {
  if (socket) {
    socket.disconnect();
    socket = null;
  }
};
```

Notification Bell Component

File: `src/components/Notifications/NotificationBell.js`

```
import { useState, useEffect } from 'react';
import { getSocket } from '../../services/socket';

const NotificationBell = () => {
  const [notifications, setNotifications] = useState([]);
  const [unreadCount, setUnreadCount] = useState(0);
  const [showDropdown, setShowDropdown] = useState(false);

  useEffect(() => {
    const socket = getSocket();

    if (socket) {
      socket.on('notification', (notification) => {
        setNotifications(prev => [notification, ...prev]);
        setUnreadCount(prev => prev + 1);
      });
    }
```

```
    return () => {
      if (socket) {
        socket.off('notification');
      }
    };
  }, []);

  return (
    <div className="relative">
      <button
        onClick={() => setShowDropdown(!showDropdown)}
        className="relative p-2 rounded-full hover:bg-gray-100"
      >
        <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24
24">
          <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
                d="M15 17h5l-1.405-1.405A2.032 2.032 0 0118 14.158V11a6.002 6.002
0 00-4-5.659V5a2 2 0 10-4 0v.341C7.67 6.165 6 8.388 6 11v3.159c0 .538-.214
1.055-.595 1.436L4 17h5m6 0v1a3 3 0 11-6 0v-1m6 0H9" />
        </svg>
        {unreadCount > 0 && (
          <span className="absolute top-0 right-0 bg-red-500 text-white text-xs
rounded-full h-5 w-5 flex items-center justify-center">
            {unreadCount}
          </span>
        )}
      </button>

      {showDropdown && (
        <div className="absolute right-0 mt-2 w-80 bg-white rounded-lg shadow-lg
z-50">
          {/* Notification list */}
        </div>
      )}
    </div>
  );
};
```

---

13. EXPORT FUNCTIONALITY

CSV Export Implementation

File: `src/utils/exportCSV.js`

```
/**
 * Export farms data to CSV file
 * @param {Array} farms - Array of farm objects
 * @param {string} username - Name of the user
 * @returns {boolean} - Success status
 */
```

```
export const exportFarmsToCSV = (farms, username = 'User') => {
  try {
    if (!farms || farms.length === 0) {
      return false;
    }

    // CSV Headers
    const headers = [
      'Farm Name',
      'Area (acres)',
      'State',
      'District',
      'Village',
      'Pincode',
      'Soil Type',
      'Irrigation Type',
      'Water Source',
      'Created Date'
    ];

    // Convert farms to CSV rows
    const rows = farms.map(farm => [
      farm.farmName || '',
      farm.area || '',
      farm.location?.state || '',
      farm.location?.district || '',
      farm.location?.village || '',
      farm.location?.pincode || '',
      farm.soil_type || '',
      farm.irrigation_type || '',
      farm.water_source || '',
      farm.createdAt ? new Date(farm.createdAt).toLocaleDateString() : ''
    ]);

    // Combine headers and rows
    const csvContent = [
      headers.join(','),
      ...rows.map(row => row.map(cell => `"${cell}"`).join(','))
    ].join('\n');

    // Create Blob and download
    const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });
    const link = document.createElement('a');
    const url = URL.createObjectURL(blob);

    link.setAttribute('href', url);
    link.setAttribute('download', `farms_${username}_${Date.now()}.csv`);
    link.style.visibility = 'hidden';

    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);

    return true;
```

```
  } catch (error) {
    console.error('Error exporting to CSV:', error);
    return false;
  }
};
```

Usage in Components

```
import { exportFarmsToCSV } from '../../utils/exportCSV';

const handleExport = () => {
  const success = exportFarmsToCSV(farms, user?.name);
  if (success) {
    alert('Export successful!');
  } else {
    alert('Export failed');
  }
};
```

---

14. PERFORMANCE OPTIMIZATION

Code Splitting & Lazy Loading

```
import { lazy, Suspense } from 'react';

const Dashboard = lazy(() => import('./components/Dashboard/Dashboard'));
const Farms = lazy(() => import('./components/Farms/Farms'));
const Crops = lazy(() => import('./components/Crops/Crops'));

function App() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/farms" element={<Farms />} />
        <Route path="/crops" element={<Crops />} />
      </Routes>
    </Suspense>
  );
}
```

Memoization

React.memo for Components

```
import { memo } from 'react';

const FarmCard = memo(({ farm, onEdit, onDelete }) => {
  return <div>...</div>;
});
```

useMemo for Expensive Calculations

```
import { useMemo } from 'react';

const filteredFarms = useMemo(() => {
  return farms.filter(farm =>
    farm.farmName.toLowerCase().includes(searchTerm.toLowerCase())
  );
}, [farms, searchTerm]);
```

useCallback for Functions

```
import { useCallback } from 'react';

const handleDelete = useCallback((id) => {
  setFarms(prev => prev.filter(farm => farm._id !== id));
}, []);
```

Image Optimization

```
// Use appropriate image formats
<img
  src="/image.webp"
  alt="Farm"
  loading="lazy"  // Lazy load images
  width="400"
  height="300"
/>
```

Debouncing Search

```
import { useState, useEffect } from 'react';

const [searchTerm, setSearchTerm] = useState('');
const [debouncedTerm, setDebouncedTerm] = useState('');

useEffect(() => {
```

```
    const timer = setTimeout(() => {
      setDebouncedTerm(searchTerm);
    }, 500);

    return () => clearTimeout(timer);
  }, [searchTerm]);

  // Use debouncedTerm for API calls
  useEffect(() => {
    if (debouncedTerm) {
      searchFarms(debouncedTerm);
    }
  }, [debouncedTerm]);
```

---

15. BUILD & DEPLOYMENT

Environment Variables

File: `.env`

```
REACT_APP_API_URL=http://localhost:5000/api
REACT_APP_SOCKET_URL=http://localhost:5000
```

Production:

```
REACT_APP_API_URL=https://your-api.com/api
REACT_APP_SOCKET_URL=https://your-api.com
```

Build Process

Development Build

```
npm start
```

- Runs on http://localhost:3000
- Hot reload enabled
- Source maps included
- Not optimized

Production Build

```
npm run build
```

- Creates `build/` folder
- Minified code
- Optimized assets
- Ready for deployment

Build Output Structure

```
build/
├── static/
│   ├── css/
│   │   └── main.[hash].css
│   ├── js/
│   │   ├── main.[hash].js
│   │   └── [chunk].[hash].js
│   └── media/
│       └── [images/fonts]
├── index.html
├── manifest.json
└── asset-manifest.json
```

Deployment Options

1. Vercel (Recommended)

```
# Install Vercel CLI
npm install -g vercel

# Deploy
cd client
vercel
```

vercel.json:

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "build",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

2. Netlify

```
# Install Netlify CLI
npm install -g netlify-cli

# Deploy
cd client
netlify deploy --prod
```

netlify.toml:

```
[build]
  command = "npm run build"
  publish = "build"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

3. Static Hosting (Nginx)

```
server {
    listen 80;
    server_name yourdomain.com;
    root /var/www/agriculture-ai/build;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api {
        proxy_pass http://localhost:5000;
    }
}
```

Performance Checklist

- ☑ Code splitting implemented
- ☑ Images lazy loaded
- ☑ API calls optimized
- ☑ Bundle size minimized
- ☑ Gzip compression enabled
- ☑ CDN for static assets
- ☑ Service worker for caching

Testing Before Deployment

```
# Build production version
npm run build

# Test locally with serve
npx serve -s build

# Check bundle size
npm run build -- --stats

# Lighthouse audit
npm install -g lighthouse
lighthouse http://localhost:3000
```

APPENDIX

Common Issues & Solutions

Issue: Blank page after deployment

Solution: Check that environment variables are set correctly and routes have proper redirects.

Issue: API calls failing

Solution: Verify REACT_APP_API_URL is correct and CORS is enabled on backend.

Issue: Translations not working

Solution: Ensure i18n is initialized before rendering components.

Issue: Socket not connecting

Solution: Check REACT_APP_SOCKET_URL and ensure WebSocket support on server.

Development Workflow

1. **Start Development:**

   ```
   cd client
   npm install
   npm start
   ```

2. **Make Changes:**

   - Edit component files
   - Add translations
   - Test in browser

3. **Test Changes:**
    - Check all three languages
    - Test responsive design
    - Verify API integration
4. **Build & Deploy:**

```
npm run build
vercel deploy
```

Best Practices Summary

1. **Component Structure:**
    - One component per file
    - Descriptive names
    - Clear separation of concerns
2. **State Management:**
    - Use Context for global state
    - useState for local state
    - Keep state minimal
3. **API Calls:**
    - Always handle errors
    - Show loading states
    - Use try-catch blocks
4. **Translation:**
    - Never hardcode text
    - Use t() function everywhere
    - Test all languages
5. **Styling:**
    - Use Tailwind utilities
    - Maintain consistency
    - Mobile-first approach
6. **Performance:**
    - Lazy load components
    - Optimize images
    - Minimize re-renders

🎯 QUICK REFERENCE

File Locations

- Components: `client/src/components/`
- API Config: `client/src/services/api.js`
- Translations: `client/src/i18n/locales/`
- Utils: `client/src/utils/`

Important Commands

```
npm start          # Development server
npm run build      # Production build
npm test           # Run tests
```

Key Hooks

- `useAuth()` - Authentication state
- `useTranslation()` - i18n translations
- `useNavigate()` - Programmatic navigation
- `useLocation()` - Current route info

API Endpoints

- `GET /api/farms` - Get all farms
- `POST /api/farms` - Create farm
- `PUT /api/farms/:id` - Update farm
- `DELETE /api/farms/:id` - Delete farm

Document Version: 1.0
Last Updated: July 18, 2026
Author: Agriculture AI Development Team

END OF FRONTEND DOCUMENTATION